

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA

Curso: Cálculo II

Docente: Ing. José Alfredo González

PROYECTO

Emulador de Volúmenes de Sólidos de Revolución

Métodos del Disco, Arandelas y Capas Cilíndricas

Informe escrito

Integrantes:

Javier Alejandro Echeverría Figueroa — Carné 3490-25-23600

Fernando Isaac Garcia Baldetti — Carné 3490-25-20014

Edison Vinicio Alvarez Pérez — Carné 3490-25-10745

Sección: B

Guatemala, 25 de octubre de 2026

Índice

1. Introducción 4
2. Objetivos 5
2.1 Objetivo general 5
2.2 Objetivos específicos 5
3. Marco teórico 6
3.1 La integral definida 6
3.2 Sólidos de revolución 6
3.3 Método del disco 6
3.4 Método de arandelas 6
3.5 Método de capas cilíndricas 7
3.6 Compatibilidad entre método y eje 7
3.7 Integración numérica: regla de Simpson 1/3 compuesta 8
3.8 Volumen acumulado 8
3.9 Intersección de curvas: método de bisección 8
4. Justificación 9
5. Resolución manual de los problemas 10
5.1 Ejercicio principal: método de arandelas 10
a) Puntos de intersección 10
b) Función superior y función inferior 10
c) Deducción de la ecuación del volumen 11
d) Cálculo del volumen 11
e) Esquema de la región, las curvas, los límites y el eje de rotación 12
5.2 Configuración 1: método del disco 14
5.3 Configuración 2: método de arandelas 15
5.4 Configuración 3: método de capas cilíndricas 16
5.5 Resumen de la resolución manual 18
6. Desarrollo del simulador 19
6.1 Tecnologías utilizadas 19
6.2 Arquitectura del programa 19
6.3 Entrada de datos 19

6.4 Validación de datos 20
6.5 Cálculo del volumen 20
6.6 Comparación con el procedimiento manual 20
6.7 Gráficas 21
6.8 Exportación de datos 21
6.9 Ejecución del programa 21
7. Validación y análisis de resultados 22
7.1 Tabla de validación 22
7.2 Análisis del error numérico 22
7.3 Análisis del volumen acumulado 23
7.4 Correspondencia con la maqueta 24
8. Conclusiones 25
9. Recomendaciones 26
10. Bibliografía 27
11. Anexos 28
Anexo A. Código fuente 28
Anexo B. Capturas de pantalla del simulador 37
Anexo C. Archivo de datos exportado 39
Anexo D. Gráficas 40

1. Introducción

El cálculo integral permite medir magnitudes que no pueden obtenerse con fórmulas geométricas elementales. Una de sus aplicaciones más visuales es el cálculo del volumen de un **sólido de revolución**: el cuerpo que se genera cuando una región plana gira 360° alrededor de una recta llamada eje de rotación. Piezas como ejes escalonados, toberas, recipientes, poleas o lentes se modelan de esta manera, por lo que el tema tiene una relación directa con el trabajo del ingeniero.

En este informe se calcula el volumen del sólido que se obtiene al girar, alrededor del eje X, la región limitada por las curvas $y = x^2 + 1$ y $y = x + 3$, utilizando el **método de arandelas**. El resultado se obtiene primero mediante un procedimiento manual completo y luego se verifica con un simulador computacional desarrollado en **Java 21 con JavaFX**, que implementa también los métodos del disco y de capas cilíndricas. Para comprobar el simulador se resuelven, además, tres configuraciones de validación: un problema de discos, uno de arandelas y uno de capas cilíndricas.

El simulador valida los datos de entrada, construye la integral del método elegido, la evalúa numéricamente con la regla de Simpson 1/3 compuesta, compara el resultado con el valor manual, genera la gráfica 2D de la región, la representación 3D del sólido y la curva de volumen acumulado, y exporta los datos a archivos CSV y TXT. Finalmente, se construyó una maqueta física con un motor de 5 V que, al hacer girar la región plana a alta velocidad, permite observar el sólido de forma tangible.

El documento se organiza así: objetivos, marco teórico, justificación, resolución manual de los cuatro problemas, desarrollo del simulador, validación y análisis de resultados, conclusiones, recomendaciones, bibliografía y anexos (código fuente, capturas de pantalla, datos exportados y gráficas).

2. Objetivos

2.1 Objetivo general

Desarrollar un simulador computacional que calcule y represente gráficamente el volumen de sólidos de revolución mediante los métodos del disco, de arandelas y de capas cilíndricas, y validar sus resultados con la resolución manual de los problemas y con una maqueta física.

2.2 Objetivos específicos

1. Determinar analíticamente los puntos de intersección de $y = x^2 + 1$ y $y = x + 3$, identificar la función superior y la inferior y deducir la integral del volumen por el método de arandelas.
2. Calcular de forma exacta el volumen del sólido generado al girar la región alrededor del eje X.
3. Resolver manualmente las tres configuraciones de validación (discos, arandelas y capas cilíndricas).
4. Implementar en Java las fórmulas de los tres métodos, la validación de datos y la integración numérica por la regla de Simpson 1/3 compuesta.
5. Generar la gráfica 2D de la región, la gráfica 3D del sólido y la gráfica del volumen acumulado, y exportar los datos a CSV y TXT.
6. Comparar los resultados manuales con los del simulador y cuantificar la diferencia porcentual.
7. Construir una maqueta física que represente el sólido, su eje de rotación, la región generadora y los límites de integración.

3. Marco teórico

3.1 La integral definida

Si una función h es continua en [a, b], se divide el intervalo en n subintervalos de ancho Δx = (b − a)/n y se elige un punto x_i^* en cada uno. La integral definida es el límite de las sumas de Riemann:

$$\int_a^b h(x)\,\mathrm{d}x = \lim_{n\to\infty} \sum_{i=1}^{n} h(x_{i*})\Delta x$$

El **Teorema Fundamental del Cálculo** permite evaluarla con una antiderivada H, es decir, una función tal que H'(x) = h(x):

$$\int_a^b h(x)\,\mathrm{d}x = H(b) - H(a)$$

La misma idea —dividir, aproximar cada pedazo y sumar— da origen a las fórmulas de volumen: si se conoce el área A(x) de cada sección transversal perpendicular al eje, el volumen es

$$V = \int_a^b A(x)\,\mathrm{d}x$$

3.2 Sólidos de revolución

Un sólido de revolución se obtiene al girar una región plana alrededor de una recta del mismo plano. Cada punto de la región que está a una distancia ρ del eje describe una circunferencia de radio ρ. Por eso, las secciones perpendiculares al eje son círculos (discos) o coronas circulares (arandelas), y las capas paralelas al eje son cilindros (Stewart et al., 2021).

3.3 Método del disco

Si la región está entre la curva y = f(x) ≥ 0 y el eje X, sobre [a, b], cada rebanada perpendicular al eje X es un disco de radio f(x) y grosor dx, con volumen $\pi[f(x)]^2\,dx$. Al sumar todas las rebanadas:

$$V = \pi \int_a^b [f(x)]^2\,\mathrm{d}x$$

3.4 Método de arandelas

Si la región está entre dos curvas f(x) ≥ g(x) ≥ 0 y no toca el eje de rotación, cada rebanada es una arandela (un disco con un agujero). El radio exterior es R(x) = f(x) y el radio interior es r(x) = g(x). El área de la arandela es $\pi(R^2 - r^2)$, por lo que:

$$V = \pi \int_a^b ([f(x)]^2 - [g(x)]^2)\,\mathrm{d}x$$

Se restan los **cuadrados** de los radios, no los radios: $\pi(R^2 - r^2) \neq \pi(R - r)^2$. Confundir estas dos expresiones es el error más frecuente al aplicar el método.

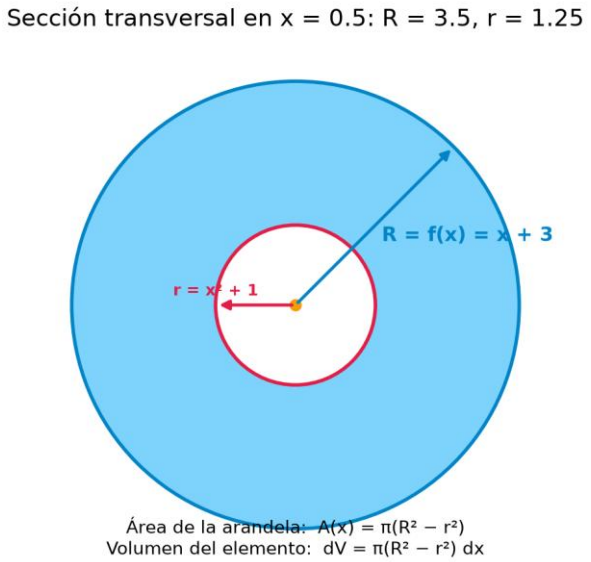


Figura 1. *Sección transversal típica (arandela) del sólido del ejercicio principal en x = 0.5.*

3.5 Método de capas cilíndricas

Cuando la región gira alrededor del eje Y y está descrita en términos de x, se descompone en franjas verticales de ancho dx. Al girar, cada franja produce una capa cilíndrica de radio x, altura f(x) − g(x) y espesor dx, cuyo volumen es $2\pi x[f(x) - g(x)]\,dx$. Para $0 \leq a < b$:

$$V = 2\pi \int_a^b x\big(f(x) - g(x)\big)\,\mathrm{d}x$$

3.6 Compatibilidad entre método y eje

Los discos y las arandelas integran secciones perpendiculares al eje de integración (x), por lo que en este proyecto se aplican con el eje X y exigen $f(x) \geq g(x) \geq 0$ para que los radios tengan sentido. Las capas integran cilindros concéntricos al eje Y con radio x, por lo que exigen $a \geq 0$ para que el radio no sea negativo.

Tabla 1. *Combinaciones de método y eje admitidas por el simulador.*

Método	Eje	Condición sobre la región	Fórmula del volumen
Disco	X	$f(x) \geq 0,\ g(x) = 0$	$V = \pi\int [f(x)]^2\,dx$
Arandelas	X	$f(x) \geq g(x) \geq 0$	$V = \pi\int ([f(x)]^2 - [g(x)]^2)\,dx$
Capas cilíndricas	Y	$0 \leq a < b,\ f(x) \geq g(x)$	$V = 2\pi\int x\,(f(x) - g(x))\,dx$

3.7 Integración numérica: regla de Simpson 1/3 compuesta

Para que el simulador funcione con cualquier f(x) y g(x), la integral se evalúa numéricamente. La regla de Simpson 1/3 aproxima la función en cada par de subintervalos por una parábola. Con n subintervalos (n par) y h = (b − a)/n:

$$\int_a^b \varphi(x)\,\mathrm{d}x \approx \frac{h}{3}[\varphi(x_0) + 4\varphi(x_1) + 2\varphi(x_2) + \cdots + 4\varphi(x_{n-1}) + \varphi(x_n)]$$

Su error de truncamiento es (Burden et al., 2017):

$$E = -\frac{(b-a)h^4}{180}\varphi^{(4)}(\xi), \quad \xi \in (a,b)$$

En consecuencia, la regla es **exacta para polinomios de grado ≤ 3** y su error disminuye con la cuarta potencia de h: al duplicar n, el error se reduce aproximadamente 16 veces. El simulador usa n = 800 para el volumen total.

3.8 Volumen acumulado

Se define el volumen acumulado como el volumen generado desde el límite inferior a hasta una posición variable x:

$$V_{\text{acum}}(x) = \int_a^x A(t)\,\mathrm{d}t, \quad V_{\text{acum}}(a) = 0, \quad V_{\text{acum}}(b) = V$$

Por el Teorema Fundamental del Cálculo, la derivada de V_{acum} es el área de la sección transversal: la pendiente de la curva en cada punto es A(x). Como A(x) ≥ 0, la función siempre es creciente y crece más rápido donde las secciones son más grandes.

$$\frac{\mathrm{d}V_{\text{acum}}}{\mathrm{d}x} = A(x)$$

3.9 Intersección de curvas: método de bisección

Los límites de integración son, en general, las abscisas donde f(x) = g(x). El simulador los obtiene buscando cambios de signo de h(x) = f(x) − g(x) en una malla fina de [−10, 10] y refinando cada uno con el método de bisección, que reduce a la mitad el intervalo que contiene la raíz en cada iteración (Chapra y Canale, 2015).

4. Justificación

El cálculo de volúmenes de revolución es un contenido fundamental de Cálculo II y una herramienta de uso cotidiano en ingeniería: permite estimar la capacidad de tanques y recipientes, la masa de piezas torneadas, la cantidad de material para fabricar un componente o el desplazamiento de un cuerpo sumergido. Sin embargo, suele ser difícil visualizar cómo una región plana se transforma en un sólido y por qué la integral describe su volumen.

Este proyecto se justifica porque aborda el problema desde tres enfoques complementarios:

- **Analítico:** la resolución manual asegura el dominio de los conceptos: intersecciones, radios, planteamiento y evaluación de la integral.
- **Computacional:** el simulador permite comprobar resultados, explorar otras funciones e intervalos en segundos y comprender la integración numérica, que es la que se usa en la práctica cuando no existe una antiderivada sencilla.
- **Físico:** la maqueta convierte el concepto abstracto en un objeto observable; al girar la región con el motor se aprecia el sólido y el hueco central que justifica el uso de arandelas.

Además, desarrollar el simulador fortalece competencias de programación, validación de datos, visualización y documentación técnica, todas necesarias en el ejercicio profesional de la ingeniería.

5. Resolución manual de los problemas

En esta sección se resuelven a mano los cuatro problemas del proyecto: el ejercicio principal y las tres configuraciones con las que se valida el simulador. Todos se resuelven con el mismo procedimiento de cuatro pasos:

1. **Límites de integración:** igualar las curvas para hallar dónde se cortan (a y b) o usar el intervalo dado.
2. **Función superior e inferior:** evaluar un punto dentro de [a, b] para saber qué curva queda arriba. De ahí salen los radios (discos y arandelas) o la altura (capas).
3. **Planteamiento:** escribir el área de la sección o el volumen de la capa y armar la integral del método.
4. **Cálculo:** desarrollar, integrar término a término, evaluar F(b) − F(a) y multiplicar por π o por 2π.

Tabla 2. *Problemas resueltos y resultados obtenidos.*

Problema	Funciones e intervalo	Método (eje)	Volumen
Ejercicio principal	f(x) = x + 3, g(x) = x² + 1, [−1, 2]	Arandelas (X)	117π/5 ≈ 73.5133 u³
Configuración 1	f(x) = √x, g(x) = 0, [0, 4]	Discos (X)	8π ≈ 25.1327 u³
Configuración 2	f(x) = √x, g(x) = x², [0, 1]	Arandelas (X)	3π/10 ≈ 0.9425 u³
Configuración 3	f(x) = 2x, g(x) = x², [0, 2]	Capas (Y)	8π/3 ≈ 8.3776 u³

5.1 Ejercicio principal: método de arandelas

Problema. Calcular el volumen del sólido generado al girar alrededor del eje X la región limitada por las curvas

$$y_1 = x^2 + 1 \quad \text{y} \quad y_2 = x + 3$$

a) Puntos de intersección

Las curvas se cortan donde tienen la misma altura, así que se igualan:

$$x^2 + 1 = x + 3$$

Se pasa todo al lado izquierdo y se factoriza buscando dos números que multiplicados den −2 y sumados den −1 (son −2 y +1):

$$x^2 - x - 2 = 0 \quad \Longrightarrow \quad (x-2)(x+1) = 0 \quad \Longrightarrow \quad x = -1, \quad x = 2$$

Sustituyendo en y = x + 3: si x = −1, entonces y = 2; si x = 2, entonces y = 5. Comprobación en la parábola: (−1)² + 1 = 2 ✓ y 2² + 1 = 5 ✓.

Puntos de intersección: $P_1(-1, 2)$ y $P_2(2, 5)$. Los límites de integración son **a = −1** y **b = 2**.

b) Función superior y función inferior

Se evalúa un punto de prueba dentro del intervalo, x = 0: la recta da 0 + 3 = 3 y la parábola da 0² + 1 = 1. Como 3 > 1, la recta está arriba. De forma general, la diferencia entre ambas es:

$$(x+3) - (x^2+1) = -x^2 + x + 2 = -(x-2)(x+1) \geq 0 \quad \text{para} \quad -1 \leq x \leq 2$$

Además, $x^2 + 1 \geq 1 > 0$: la parábola nunca baja de 1, de modo que la región está completamente por encima del eje X sin tocarlo. Al girar queda un hueco en el centro del sólido, por lo que corresponde usar **arandelas**.

Tabla 3. *Identificación de radios del ejercicio principal.*

Curva	Posición	Papel en el método
f(x) = x + 3	Superior	Radio exterior R(x) = x + 3
g(x) = x² + 1	Inferior	Radio interior r(x) = x² + 1
Condición f(x) ≥ g(x) ≥ 0 en [−1, 2]	—	Se cumple, porque g(x) ≥ 1

c) Deducción de la ecuación del volumen

Se corta el sólido con un plano perpendicular al eje X en la posición x. La sección es una arandela: un círculo de radio R(x) = x + 3 con un hueco de radio r(x) = x² + 1. Su área es la del círculo grande menos la del hueco:

$$A(x) = \pi R^2 - \pi r^2 = \pi[(x+3)^2 - (x^2+1)^2]$$

Una rebanada de grosor dx tiene volumen dV = A(x) dx. Sumando todas las rebanadas desde x = −1 hasta x = 2 se obtiene la ecuación del volumen:

$$V = \pi \int_{-1}^{2} [(x+3)^2 - (x^2+1)^2]\,\mathrm{d}x$$

Se desarrollan los cuadrados y se restan, cuidando los signos:

$$(x+3)^2 = x^2 + 6x + 9 \qquad (x^2+1)^2 = x^4 + 2x^2 + 1$$

$$(x^2 + 6x + 9) - (x^4 + 2x^2 + 1) = -x^4 - x^2 + 6x + 8$$

$$V = \pi \int_{-1}^{2} (-x^4 - x^2 + 6x + 8)\,\mathrm{d}x$$

d) Cálculo del volumen

Se integra término a término con la regla de la potencia, $\int x^n\,dx = x^{n+1}/(n+1)$:

$$F(x) = -\frac{x^5}{5} - \frac{x^3}{3} + 3x^2 + 8x$$

Se evalúa en el límite superior, x = 2, usando denominador común 15:

$$F(2) = -\frac{32}{5} - \frac{8}{3} + 12 + 16 = \frac{-96 - 40 + 420}{15} = \frac{284}{15}$$

Se evalúa en el límite inferior, x = −1:

$$F(-1) = \frac{1}{5} + \frac{1}{3} + 3 - 8 = \frac{3 + 5 + 45 - 120}{15} = -\frac{67}{15}$$

Por el Teorema Fundamental del Cálculo se restan ambos valores y se multiplica por π:

$$V = \pi[F(2) - F(-1)] = \pi\left(\frac{284}{15} + \frac{67}{15}\right) = \frac{351\pi}{15} = \frac{117\pi}{5}$$

$$\boxed{V = \frac{117\pi}{5} \approx 73.5133 \text{ u}^3}$$

Comprobación. Teorema de Pappus: V = 2π·ȳ·Área, donde ȳ es la distancia del centroide de la región al eje. El área de la región es

$$\text{Área} = \int_{-1}^{2} (-x^2 + x + 2)\, \mathrm{d}x = \frac{9}{2} = 4.5 \text{ u}^2 \quad \Longrightarrow \quad \bar{y} = \frac{V}{2\pi \cdot 4.5} = \frac{23.4\pi}{9\pi} = 2.6$$

El valor ȳ = 2.6 es coherente, porque la región se encuentra entre y = 1 y y = 5. El simulador obtiene 73.5133 u³, el mismo valor.

Error común: restar los radios y luego elevar al cuadrado. (R − r)² no es lo mismo que R² − r²: con R = 5 y r = 3, (5 − 3)² = 4, pero 5² − 3² = 16.

Volumen acumulado (forma cerrada)

Con la misma antiderivada, el volumen generado desde a = −1 hasta una posición x es:

$$V_{\text{acum}}(x) = \pi[F(x) - F(-1)] = \pi\left(-\frac{x^5}{5} - \frac{x^3}{3} + 3x^2 + 8x + \frac{67}{15}\right), \quad -1 \le x \le 2$$

Por ejemplo, $V_{acum}(0.5) = \pi(9.16875) \approx 28.8045$ u³ y $V_{acum}(2) = 117\pi/5$. La derivada $A(x) = \pi(-x^4 - x^2 + 6x + 8)$ cumple $A'(x) = \pi(-4x^3 - 2x + 6) = 0$ en x = 1, por lo que la sección más grande es $A(1) = 12\pi \approx 37.70$ u². En x = 1 la curva $V_{acum}(x)$ tiene su punto de inflexión: es donde crece con mayor rapidez.

e) Esquema de la región, las curvas, los límites y el eje de rotación

La Figura 2 muestra la región R limitada por la recta (radio exterior) y la parábola (radio interior), los límites x = −1 y x = 2 y el eje de rotación y = 0. La Figura 3 muestra el sólido que se obtiene al girarla y la Figura 4 el sólido cortado por la mitad, donde se aprecia el hueco que deja g(x).

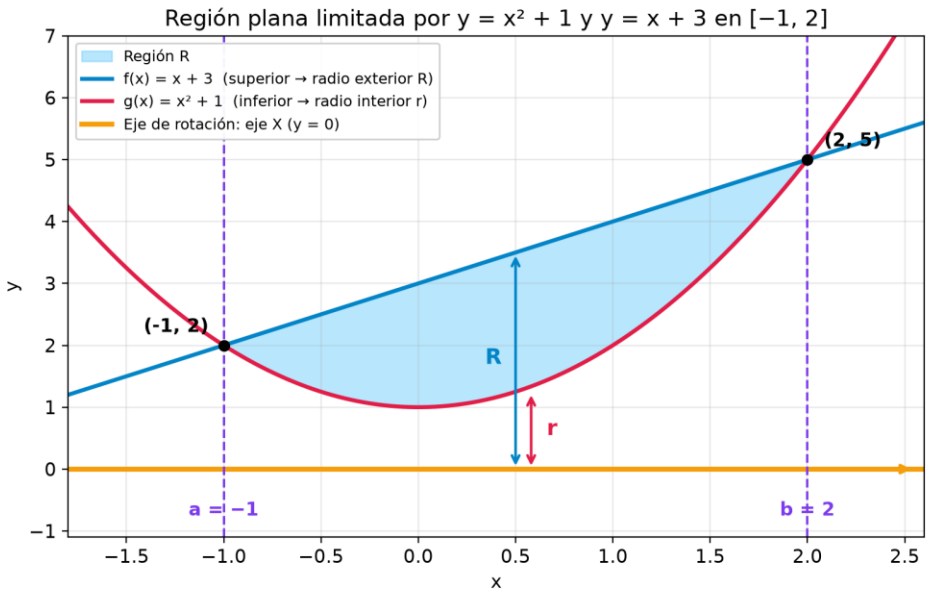


Figura 2. *Región R limitada por y = x + 3 y y = x² + 1 en [−1, 2]; eje de rotación y = 0.*

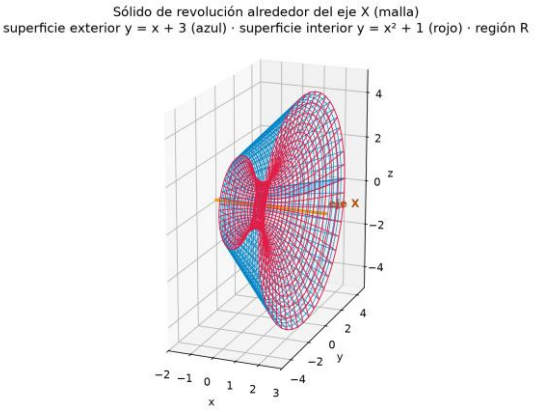


Figura 3. *Sólido de revolución: superficie exterior (azul) e interior, el hueco (rojo).*

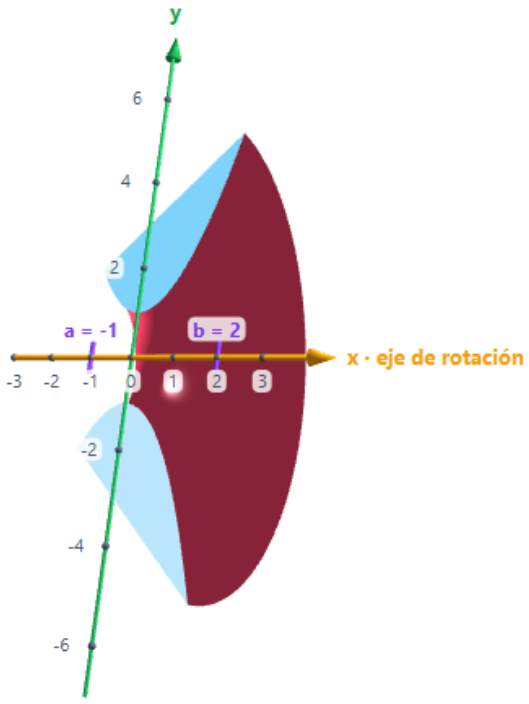


Figura 4. *Sólido cortado por la mitad en el simulador: región R arriba, su reflejo abajo y el hueco central.*

5.2 Configuración 1: método del disco

Problema. Calcular el volumen del sólido generado al girar alrededor del eje X la región bajo f(x) = √x, con g(x) = 0, en el intervalo [0, 4].

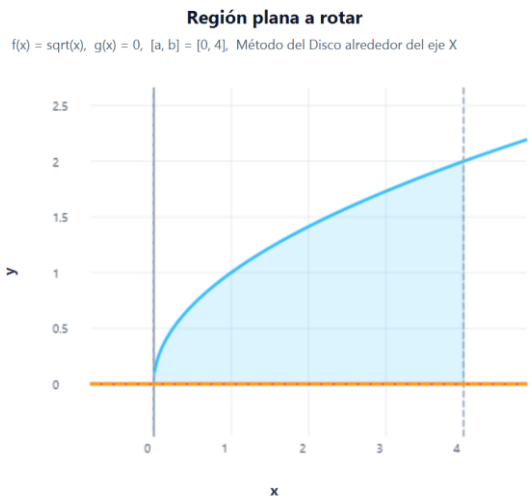


Figura 5. *Región bajo y = √x entre x = 0 y x = 4; el eje de rotación es su borde inferior.*

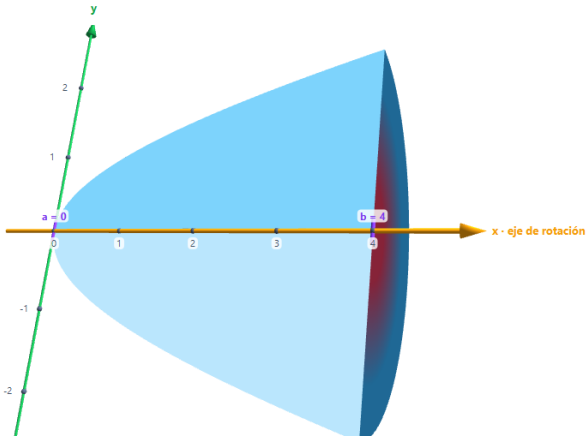


Figura 6. *Sólido cortado por la mitad: paraboloide macizo, sin hueco, porque la región toca el eje.*

Paso 1 · Límites de integración

El intervalo es dato: a = 0 y b = 4. Además, √x solo está definida para x ≥ 0, así que a = 0 es el menor valor posible. En los extremos: f(0) = 0 y f(4) = √4 = 2.

Paso 2 · Radio

g(x) = 0 es el propio eje X: la región toca el eje en todo el intervalo. No hay hueco, así que cada corte es un disco lleno de radio:

$$R(x) = f(x) = \sqrt{x}$$

Paso 3 · Planteamiento de la integral

El área de cada disco es πR². Al elevar la raíz al cuadrado, la expresión se simplifica:

$$A(x) = \pi\left(\sqrt{x}\right)^2 = \pi x \quad \Longrightarrow \quad V = \pi \int_0^4 x \, \mathrm{d}x$$

Paso 4 · Cálculo del volumen

$$V = \pi \left[\frac{x^2}{2}\right]_0^4 = \pi\left(\frac{16}{2} - 0\right) = 8\pi$$

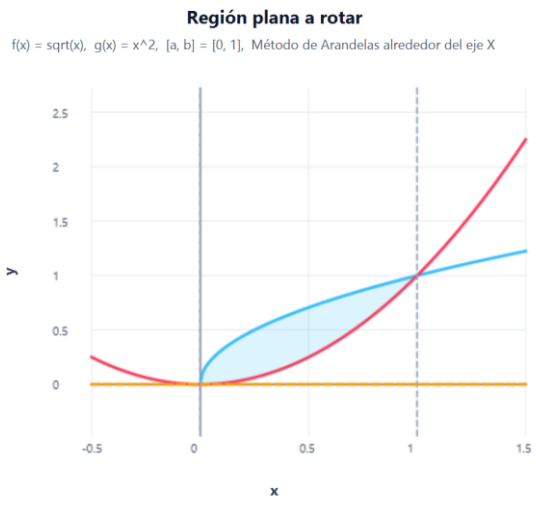
$$\boxed{V = 8\pi \approx 25.1327 \text{ u}^3}$$

Comprobación geométrica: el sólido es un paraboloide de altura 4 y radio 2. Un paraboloide ocupa la mitad del cilindro que lo contiene: $\pi \cdot 2^2 \cdot 4/2 = 8\pi$ ✓. El simulador obtiene 25.1327 u³.

Error común: olvidar que $(\sqrt{x})^2 = x$ y dejar $\sqrt{x}$ dentro de la integral, o integrar f(x) en lugar de $[f(x)]^2$. Sin el cuadrado se estaría calculando un área, no un volumen.

5.3 Configuración 2: método de arandelas

Problema. Calcular el volumen del sólido generado al girar alrededor del eje X la región entre $f(x) = \sqrt{x}$ y $g(x) = x^2$ en el intervalo [0, 1].



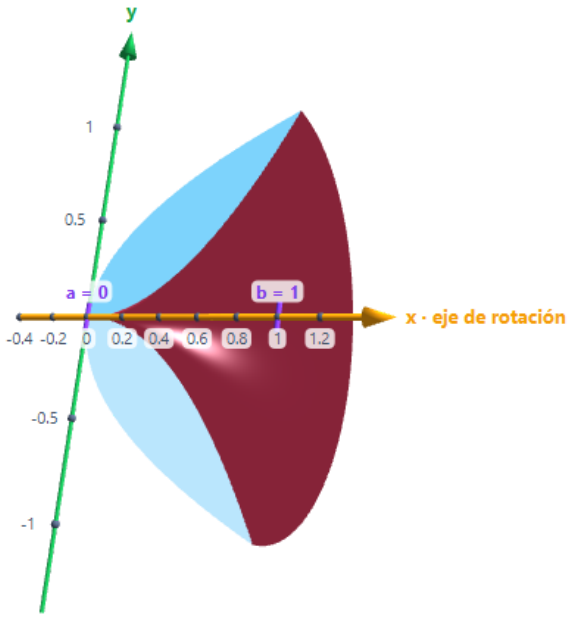


Figura 7. *Región entre $y = \sqrt{x}$ (arriba) y $y = x^2$ (abajo); se cortan en x = 0 y x = 1.*

Figura 8. *Sólido cortado por la mitad: superficie de $\sqrt{x}$ por fuera y hueco de x^2 por dentro.*

Paso 1 · Puntos de intersección

Se igualan las curvas y se elevan ambos lados al cuadrado para eliminar la raíz:

$$\sqrt{x} = x^2 \quad \Longrightarrow \quad x = x^4 \quad \Longrightarrow \quad x^4 - x = 0 \quad \Longrightarrow \quad x(x^3 - 1) = 0$$

Las soluciones son x = 0 y $x^3 = 1$, es decir, x = 1. Los puntos de corte son (0, 0) y (1, 1), que coinciden con el intervalo dado: **a = 0** y **b = 1**.

Paso 2 · Función superior e inferior (radios)

Se prueba x = 0.25: $\sqrt{0.25} = 0.5$ y $0.25^2 = 0.0625$. La raíz queda arriba. Ambas funciones son ≥ 0 en [0, 1], así que se cumple $f(x) \geq g(x) \geq 0$ y entre las curvas queda un hueco: es una arandela.

Tabla 4. *Identificación de radios de la configuración 2.*

Curva	Posición	Papel en el método
f(x) = √x	Superior	Radio exterior R(x) = √x
g(x) = x²	Inferior	Radio interior r(x) = x²

Paso 3 · Planteamiento de la integral

$$A(x) = \pi\left[\left(\sqrt{x}\right)^2 - (x^2)^2\right] = \pi(x - x^4) \quad \Longrightarrow \quad V = \pi\int_0^1 (x - x^4)\,\mathrm{d}x$$

Paso 4 · Cálculo del volumen

$$V = \pi\left[\frac{x^2}{2} - \frac{x^5}{5}\right]_0^1 = \pi\left(\frac{1}{2} - \frac{1}{5}\right) = \frac{3\pi}{10}$$

$$V = \frac{3\pi}{10} \approx 0.9425\ \mathrm{u}^3$$

Comprobación. Teorema de Pappus. El área de la región es ∫₀¹ (√x − x²) dx = 2/3 − 1/3 = 1/3 u² y su centroide está a ȳ = 3·½·(3/10) = 0.45 del eje, de modo que V = 2π·0.45·(1/3) = 0.3π ✓. El simulador obtiene 0.942478 u³.

Error común: escribir (x²)² = x³ en lugar de x⁴. La regla es (xᵐ)ⁿ = xᵐⁿ, así que (x²)² = x⁴. También es común invertir los radios: el exterior es √x, la curva que queda arriba.

5.4 Configuración 3: método de capas cilíndricas

Problema. Calcular el volumen del sólido generado al girar alrededor del eje Y la región entre f(x) = 2x y g(x) = x² en el intervalo [0, 2].

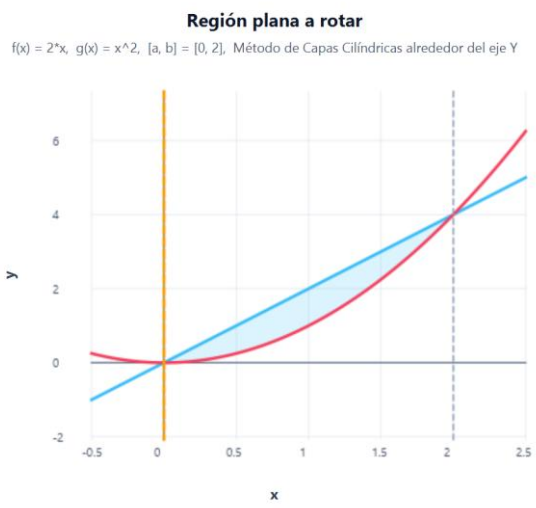


Figura 9. *Región entre y = 2x (arriba) y y = x² (abajo); el eje de rotación es el eje Y.*

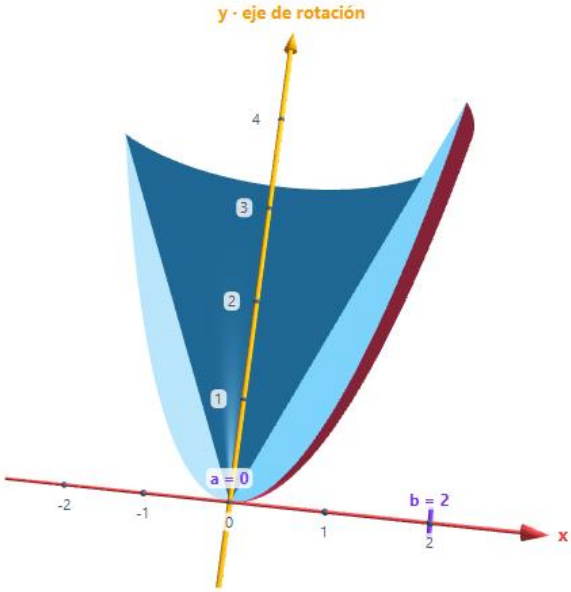


Figura 10. *Sólido cortado por la mitad: copa entre el paraboloide y = x² y el cono y = 2x.*

Paso 1 · Puntos de intersección

$$2x = x^2 \quad \Longrightarrow \quad x^2 - 2x = 0 \quad \Longrightarrow \quad x(x-2) = 0 \quad \Longrightarrow \quad x = 0, \;\; x = 2$$

Los puntos de corte son (0, 0) y (2, 4), así que **a = 0** y **b = 2**. Para capas se exige 0 ≤ a < b, y aquí a = 0 ✓.

Paso 2 · Radio y altura de cada capa

Se prueba x = 1: 2x = 2 y x² = 1, por lo que la recta queda arriba. Al girar alrededor del eje Y, cada rectángulo vertical de la región forma un tubo (una capa cilíndrica). Si se desenrolla, es una lámina de largo 2πx, alto h(x) y grosor dx.

Tabla 5. *Elementos de la capa cilíndrica de la configuración 3.*

Elemento de la capa	Valor
Radio (distancia al eje Y)	x
Altura	h(x) = f(x) − g(x) = 2x − x²
Grosor	dx

Paso 3 · Planteamiento de la integral

$$\mathrm{d}V = 2\pi \cdot (\text{radio}) \cdot (\text{altura}) \cdot \mathrm{d}x = 2\pi x(2x - x^2)\,\mathrm{d}x \quad \Longrightarrow \quad V = 2\pi \int_0^2 (2x^2 - x^3)\,\mathrm{d}x$$

Paso 4 · Cálculo del volumen

$$V = 2\pi\left[\frac{2x^3}{3} - \frac{x^4}{4}\right]_0^2 = 2\pi\left(\frac{16}{3} - 4\right) = 2\pi \cdot \frac{4}{3} = \frac{8\pi}{3}$$

$$V = \frac{8\pi}{3} \approx 8.3776\ \mathrm{u}^3$$

Comprobación con arandelas en y: el mismo sólido se puede calcular con cortes horizontales (arandelas en y). Despejando, la recta es x = y/2 y la parábola es x = √y, con y de 0 a 4. El radio exterior es √y y el interior y/2:

$$V = \pi\int_0^4 \left(y - \frac{y^2}{4}\right) dy = \pi\left(8 - \frac{16}{3}\right) = \frac{8\pi}{3}\ \checkmark$$

El simulador obtiene 8.3776 u³.

Error común: usar f(x) como radio. En capas el radio es la distancia al eje de giro, x; las funciones solo dan la altura, f(x) − g(x). También es frecuente olvidar el factor 2π.

5.5 Resumen de la resolución manual

Tabla 6. *Resultados de la resolución manual.*

Problema	Integral planteada	Valor exacto	Valor decimal (u³)
Principal	$\pi \int_{-1}^{2} [(x+3)^2 - (x^2+1)^2]\, dx$	117π/5	73.5133
Configuración 1	$\pi \int_0^4 x\, dx$	8π	25.1327
Configuración 2	$\pi \int_0^1 (x - x^4)\, dx$	3π/10	0.9425
Configuración 3	$2\pi \int_0^2 x(2x - x^2)\, dx$	8π/3	8.3776

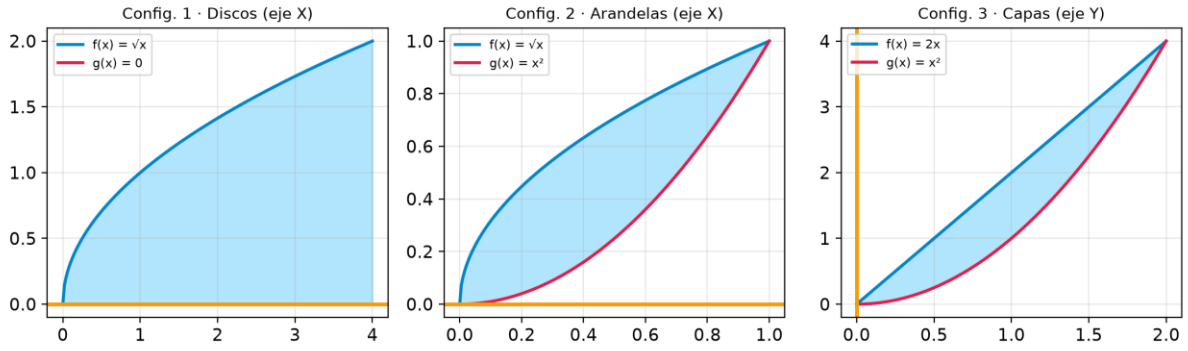


Figura 11. *Regiones de las tres configuraciones de validación (eje de rotación en naranja).*

6. Desarrollo del simulador

6.1 Tecnologías utilizadas

El simulador se desarrolló en Java porque produce un ejecutable de escritorio independiente, que no requiere instalar Java ni R en la computadora donde se presenta, y porque JavaFX incluye un motor 3D con el que se construyó la representación del sólido desde cero.

Tabla 7. *Tecnologías del simulador.*

Herramienta	Versión	Uso en el proyecto
Java (JDK)	21 LTS	Lenguaje de programación del simulador
JavaFX	21.0.4	Interfaz gráfica, gráficas 2D (Canvas) y escena 3D (MeshView, PerspectiveCamera)
exp4j	0.4.8	Interpretación de las expresiones f(x) y g(x) escritas por el usuario
Maven + jpackage	—	Compilación, empaquetado en JAR y generación del ejecutable .exe

6.2 Arquitectura del programa

El código se divide en cuatro paquetes: model (datos de los problemas), engine (cálculo), export (archivos) y ui (interfaz). La Tabla 8 resume las clases principales y la función de la plantilla base en R a la que corresponde cada una.

Tabla 8. *Clases principales del simulador.*

Clase (paquete)	Responsabilidad	Equivalente en R
FunctionEvaluator (engine)	Convierte el texto de f(x) y g(x) en funciones evaluables	f(), g()
Validator (engine)	Reglas de validación de los datos de entrada	validar_datos()
VolumeCalculator (engine)	Integrando de cada método, regla de Simpson y volumen acumulado	calcular_volumen(), calcular_volumen_acumulado()
IntersectionFinder (engine)	Raíces de f(x) − g(x) por bisección	— (adicional)
RevolutionMeshBuilder (engine)	Malla triangular 3D del sólido	graficar_solido_3d()
DataExporter, ImageExporter (export)	Exportación CSV/TXT y PNG	exportar_datos()
MainView, Region2DCanvas, AccumCanvas, Solid3DView (ui)	Interfaz, gráfica 2D, gráfica de volumen acumulado y vista 3D	graficar_region(), graficar_volumen_acumulado()
Presets (model)	Ejercicio principal y configuraciones 1 a 3 precargadas	—

6.3 Entrada de datos

Las funciones f(x) (superior) y g(x) (inferior) están definidas en el código fuente como configuraciones predefinidas (clase Presets) y también pueden modificarse en la interfaz; se aceptan expresiones como x^2 + 1, sqrt(x), sin(x), exp(x) o pi. El usuario ingresa los límites a y b —o los calcula con el botón

«Calcular intersecciones f(x) = g(x)»—, selecciona el método de cálculo y el eje de rotación, e ingresa el volumen obtenido manualmente (acepta 73.5133, 73,5133 o la forma exacta 117*pi/5).

6.4 Validación de datos

Antes de calcular, la clase Validator revisa los datos y muestra mensajes claros. Si existe algún error, el volumen no se calcula.

Tabla 9. *Validaciones implementadas.*

Regla verificada	Mensaje mostrado
a y b son números	Los límites de integración a y b deben ser números válidos.
a < b	Debe cumplirse que a < b.
Discos y arandelas solo con eje X	Los métodos de discos y arandelas deben ser alrededor del eje X.
Capas solo con eje Y	El método de capas cilíndricas debe ser alrededor del eje Y.
Capas: 0 ≤ a	Para capas cilíndricas alrededor del eje Y debe cumplirse 0 <= a < b.
f y g definidas en [a, b] (201 puntos)	Las funciones no se pudieron evaluar en todo el intervalo [a, b].
f(x) ≥ g(x) en [a, b]	Debe cumplirse f(x) >= g(x) en todo el intervalo [a, b].
Discos y arandelas: g(x) ≥ 0	Para discos y arandelas, debe cumplirse f(x) >= g(x) >= 0 en [a, b].

Además, en la interfaz el botón del eje incompatible se deshabilita al seleccionar el método, de modo que una combinación incorrecta no puede elegirse por accidente; si se fuerza, el validador la rechaza.

6.5 Cálculo del volumen

El método VolumeCalculator.buildIntegrand construye el integrando según el método elegido, con exactamente las mismas fórmulas del marco teórico y de la resolución manual:

Tabla 10. *Integrandos del código.*

Método	Integrando implementado en Java
DISCOS	$\pi \cdot f(x)^2$
ARANDELAS	$\pi \cdot (f(x)^2 - g(x)^2)$
CAPAS	$2\pi \cdot x \cdot (f(x) - g(x))$

Ese integrando se evalúa con la regla de Simpson 1/3 compuesta (VolumeCalculator.simpsonIntegrate) con n = 800 subintervalos. El resultado se muestra con cuatro decimales y unidades cúbicas (u^3), junto con la integral planteada; por ejemplo: «73.5133 u^3 — $V = \pi \int ([f(x)]^2 - [g(x)]^2)\, dx$ en [−1.00, 2.00]».

6.6 Comparación con el procedimiento manual

El usuario escribe el volumen manual y el programa calcula la diferencia porcentual respecto del valor del simulador:

$$\text{Diferencia } \% = \frac{|V_{\text{sim}} - V_{\text{manual}}|}{|V_{\text{sim}}|} \times 100$$

La concordancia se clasifica así: menor a 0.5 %, excelente concordancia; menor a 2 %, diferencia pequeña; menor a 5 %, diferencia moderada; 5 % o más, diferencia significativa. También se contempla el caso en que ambos resultados son cero.

6.7 Gráficas

- **Gráfica 2D de la región:** curva superior f(x) en azul, curva inferior g(x) en rosa, región sombreada, líneas discontinuas en a y b, eje de rotación resaltado en naranja, título, ejes rotulados y escala automática.
- **Gráfica 3D del sólido:** la clase RevolutionMeshBuilder construye un perfil cerrado (curva exterior → tapa → curva interior → tapa) y lo barre alrededor del eje para formar una malla triangular. La vista permite rotar con el ratón, hacer zoom, mostrar un corte transversal en una posición x elegida (con el valor de A(x)), cortar el sólido por la mitad y animar la generación del sólido.
- **Volumen acumulado:** se grafica $V_{acum}(x)$ desde a hasta b con 151 puntos, cada uno calculado con Simpson (n = 200) desde a hasta x, y se marca el volumen total.

Todas las gráficas se pueden exportar a PNG con el botón «Exportar gráficas PNG».

6.8 Exportación de datos

Los botones «Exportar CSV» y «Exportar TXT» generan un archivo con las columnas x, f(x), g(x) y $V_{acum}(x)$ (151 filas, cuatro decimales). El CSV usa comas y el TXT tabulaciones. Un extracto se presenta en el Anexo C.

6.9 Ejecución del programa

- **Ejecutable de Windows:** abrir SimuladorSolidos\SimuladorSolidos.exe (no requiere Java instalado; se debe copiar la carpeta completa).
- **JAR:** java -jar simulador-solidos-revolucion-1.0.0-shaded.jar (requiere Java 21).
- **Desde el código fuente:** mvn javafx:run (ejecutar) y mvn package (compilar).

7. Validación y análisis de resultados

7.1 Tabla de validación

La Tabla 11 compara el volumen calculado a mano con el del simulador. El valor manual se ingresa en el programa redondeado a cuatro decimales, tal como se obtuvo en la Sección 5.

Tabla 11. *Comparación entre la resolución manual y el simulador.*

Config.	Funciones	Intervalo	Método	Manual (u^3)	Simulador (u^3)	Error %
Principal	$f = x + 3,\ g = x^2 + 1$	[−1, 2]	Arandelas (X)	73.5133	73.5133	0.0000
1	$f = \sqrt{x},\ g = 0$	[0, 4]	Discos (X)	25.1327	25.1327	0.0002
2	$f = \sqrt{x},\ g = x^2$	[0, 1]	Arandelas (X)	0.9425	0.9425	0.0024
3	$f = 2x,\ g = x^2$	[0, 2]	Capas (Y)	8.3776	8.3776	0.0002

En las cuatro configuraciones la diferencia es menor a 0.003 %, muy por debajo del umbral de 0.5 % que el programa considera excelente concordancia. Estas pequeñas diferencias se deben únicamente al redondeo del valor manual a cuatro decimales: por ejemplo, $3\pi/10 = 0.942478$ se escribe como 0.9425. El simulador muestra la misma tabla en la pestaña «Tabla de Validación» con el indicador «4/4 configuraciones validadas» (Anexo B).

7.2 Análisis del error numérico

Si se compara el resultado del simulador con el valor exacto (sin redondear), la diferencia que queda es el error de truncamiento de la regla de Simpson. En el ejercicio principal el integrando $\pi(-x^4 - x^2 + 6x + 8)$ es de grado 4 y su cuarta derivada es constante, -24π, por lo que el error teórico puede calcularse exactamente:

$$|E| = \frac{(b-a)h^4 \cdot 24\pi}{180} = \frac{3 \cdot \left(\frac{3}{800}\right)^4 \cdot 24\pi}{180} \approx 2.49 \times 10^{-10}\ \mathrm{u}^3$$

El valor observado es $|73.5132680937526 - 117\pi/5| \approx 2.49 \times 10^{-10}\ u^3$, que coincide con la predicción teórica. En las configuraciones 1 y 3 los integrandos (πx y $2\pi x(2x - x^2)$) son polinomios de grado ≤ 3, para los cuales Simpson es exacto: la diferencia es solo de redondeo de la computadora ($\approx 10^{-15}$). En la configuración 2, $\pi(x - x^4)$ es de grado 4 y el error es del orden de 10^{-10}.

La Tabla 12 muestra la convergencia del método en el ejercicio principal al aumentar el número de subintervalos n:

Tabla 12. *Convergencia de la regla de Simpson 1/3 compuesta (ejercicio principal).*

n	h	V aproximado (u^3)	Error absoluto	Error relativo
4	0.75000	73.11566027	3.98×10^{-1}	5.41×10^{-1} %
10	0.30000	73.50308933	1.02×10^{-2}	1.38×10^{-2} %
50	0.06000	73.51325181	1.63×10^{-5}	2.22×10^{-5} %
200	0.01500	73.51326803	6.36×10^{-8}	8.65×10^{-8} %
800	0.00375	73.51326809	2.49×10^{-10}	3.38×10^{-10} %

Al multiplicar n por 4 (de 200 a 800), el error se divide entre $4^4 = 256$, exactamente lo que predice el término h^4. Con n = 800 el resultado es correcto en todas las cifras significativas que se reportan.

7.3 Análisis del volumen acumulado

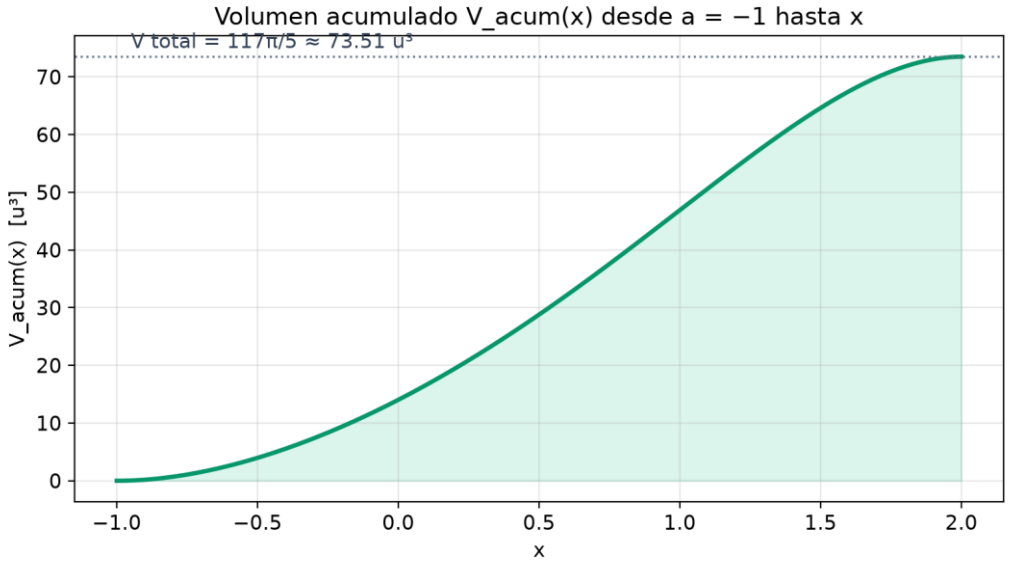


Figura 12. *Volumen acumulado V_acum(x) del ejercicio principal.*

La curva parte de $V_{acum}(-1) = 0$ y termina en $V_{acum}(2) = 73.5133$ u³, el volumen total. Es creciente en todo el intervalo porque $A(x) \geq 0$. Al inicio crece despacio (en x = −1 las curvas se cortan y A(−1) = 0), alcanza su mayor pendiente en x = 1 —donde la arandela tiene la mayor área, $A(1) = 12\pi \approx 37.70$ u²— y vuelve a aplanarse cerca de x = 2, donde las curvas se cortan de nuevo y A(2) = 0. Esa forma de «S» confirma que la derivada del volumen acumulado es el área de la sección.

Tabla 13. *Valores seleccionados del archivo exportado datos_solido.csv.*

x	f(x)	g(x)	V_acum(x) (u³)
−1.0	2.0000	2.0000	0.0000
−0.5	2.5000	1.2500	3.9728
0.0	3.0000	1.0000	14.0324
0.5	3.5000	1.2500	28.8045
1.0	4.0000	2.0000	46.9145
1.5	4.5000	3.2500	64.6317
2.0	5.0000	5.0000	73.5133

A la mitad del intervalo (x = 0.5) solo se ha acumulado el 39 % del volumen (28.80 de 73.51 u³), porque las arandelas de la segunda mitad son más grandes que las de la primera.

7.4 Correspondencia con la maqueta

La maqueta reproduce el sólido a escala 1 u = 2 cm. En esa escala las longitudes se multiplican por 2 y los volúmenes por $2^3 = 8$, de modo que el sólido físico equivale a $73.5133 \times 8 \approx 588.1$ cm³. Sus dimensiones principales son: largo 6 cm (de a = −1 a b = 2), radio exterior de 4 cm en x = −1 y de 10 cm en x = 2 (diámetro máximo 20 cm) y radio interior mínimo de 2 cm en x = 0.

8. Conclusiones

1. Las curvas $y = x^2 + 1$ y $y = x + 3$ se cortan en $(-1, 2)$ y $(2, 5)$. En $[-1, 2]$ la recta es la función superior (radio exterior) y la parábola la inferior (radio interior). Como la región no toca el eje X, el sólido tiene un hueco y el método de arandelas es el adecuado.
2. El volumen del sólido es $V = \pi\int_{-1}^{2} [(x + 3)^2 - (x^2 + 1)^2]\, dx = 117\pi/5 \approx 73.5133\ u^3$, resultado verificado además con el teorema de Pappus.
3. Las tres configuraciones de validación dieron $8\pi \approx 25.1327\ u^3$ (discos), $3\pi/10 \approx 0.9425\ u^3$ (arandelas) y $8\pi/3 \approx 8.3776\ u^3$ (capas cilíndricas), cada una comprobada por un segundo camino (geometría, Pappus o arandelas en y).
4. El simulador reproduce los cuatro resultados manuales con una diferencia menor a 0.003 %, atribuible al redondeo del valor manual, lo que valida tanto las fórmulas implementadas como el procedimiento manual.
5. El error propio del simulador se explica por el término de truncamiento de la regla de Simpson, proporcional a h^4; la regla es exacta para polinomios de grado ≤ 3 y con n = 800 el error es del orden de $10^{-10}\ u^3$.
6. La gráfica de volumen acumulado confirma el Teorema Fundamental del Cálculo: su pendiente es el área de la sección transversal, máxima en x = 1, donde la curva presenta su punto de inflexión.
7. La maqueta permitió observar físicamente el sólido y su hueco central; su escala (1 u = 2 cm) relaciona directamente las dimensiones físicas con el modelo matemático y con la gráfica 3D del simulador.

9. Recomendaciones

1. Verificar siempre cuál función es la superior evaluando un punto de prueba dentro del intervalo, y restar los cuadrados de los radios ($R^2 - r^2$), no el cuadrado de la diferencia.
2. Antes de aplicar un método, comprobar su compatibilidad con el eje y las condiciones de la región ($f \geq g \geq 0$ para el eje X; $0 \leq a$ para capas alrededor del eje Y).
3. Cuando el integrando no sea suave (por ejemplo, con $\sqrt{x}$ cerca de 0), aumentar el número de subintervalos o comparar dos valores de n para estimar el error numérico.
4. Ampliar el simulador para permitir ejes de rotación desplazados ($y = k$, $x = h$) y el método de capas alrededor del eje X, integrando respecto de y.
5. En la maqueta, usar materiales ligeros, centrar con precisión la región sobre el eje y balancearla antes de encender el motor para evitar vibraciones; operarla con protección y a distancia segura.
6. Documentar el código y volver a verificar las configuraciones de validación después de cada modificación para detectar errores de forma temprana.

10. Bibliografía

Burden, R. L., Faires, J. D., & Burden, A. M. (2017). *Análisis numérico* (10.ª ed.). Cengage Learning.

Chapra, S. C., & Canale, R. P. (2015). *Métodos numéricos para ingenieros* (7.ª ed.). McGraw-Hill Education.

Larson, R., & Edwards, B. H. (2016). *Cálculo* (10.ª ed., Tomo I). Cengage Learning.

Stewart, J., Clegg, D. K., & Watson, S. (2021). *Cálculo de una variable: trascendentes tempranas* (9.ª ed.). Cengage Learning.

Thomas, G. B., Weir, M. D., & Hass, J. (2015). *Cálculo: una variable* (13.ª ed.). Pearson Educación.

Asta, F. (2016). *exp4j: Mathematical expression evaluator* (versión 0.4.8) [Software]. https://www.objecthunter.net/exp4j/

OpenJFX. (2024). *JavaFX 21 documentation*. https://openjfx.io/

Oracle. (2023). *Java SE 21 documentation*. https://docs.oracle.com/en/java/javase/21/

11. Anexos

Anexo A. Código fuente

Se incluye el código completo del motor de cálculo (paquetes model, engine y export), que contiene las ecuaciones, la validación, la integración numérica, la malla 3D y la exportación. Las clases de interfaz gráfica (paquete ui: MainView, Region2DCanvas, AccumCanvas y Solid3DView) se entregan en la carpeta del código fuente src/main/java/com/umg/calculo/.

model/Metodo.java

```
package com.umg.calculo.model;

public enum Metodo {
    DISCOS, ARANDELAS, CAPAS;

    public String etiqueta() {
        return switch (this) {
            case DISCOS -> "Método del Disco";
            case ARANDELAS -> "Método de Arandelas";
            case CAPAS -> "Método de Capas Cilíndricas";
        };
    }
}
```

model/Eje.java

```
package com.umg.calculo.model;

public enum Eje { X, Y }
```

model/PresetConfig.java

```
package com.umg.calculo.model;

/** One of the assignment's predefined problem configurations (main exercise + 3 validation cases). */
public record PresetConfig(
        String id,
        String nombre,
        String f,
        String g,
        double a,
        double b,
        Metodo metodo,
        Eje eje,
        double volumenManual,
        String descripcion
) {}
```

model/Presets.java

```
package com.umg.calculo.model;

import java.util.List;

/** The assignment's fixed test cases: the main exercise plus the 3 required validation configs (Section 13).
*/
public final class Presets {

    private Presets() {}

    public static final PresetConfig MAIN_PROJECT = new PresetConfig(
            "main_project",
```

```
            "Ejercicio Principal (Arandelas - Eje X)",
            "x + 3", "x^2 + 1",
            -1, 2, Metodo.ARANDELAS, Eje.X,
            73.5133,
            "Sólido generado al girar y = x^2 + 1 e y = x + 3 en [-1, 2] alrededor del eje X."
    );

    public static final PresetConfig CONFIG_1 = new PresetConfig(
            "config_1",
            "Configuración 1 (Discos - Eje X)",
            "sqrt(x)", "0",
            0, 4, Metodo.DISCOS, Eje.X,
            25.1327, // pi * [x^2/2]_0^4 = 8*pi
            "f(x) = sqrt(x), g(x) = 0 en [0, 4] rotado en el Eje X."
    );

    public static final PresetConfig CONFIG_2 = new PresetConfig(
            "config_2",
            "Configuración 2 (Arandelas - Eje X)",
            "sqrt(x)", "x^2",
            0, 1, Metodo.ARANDELAS, Eje.X,
            0.9425, // pi * int_0^1 (x - x^4) dx = 0.3*pi
            "f(x) = sqrt(x), g(x) = x^2 en [0, 1] rotado en el Eje X."
    );

    public static final PresetConfig CONFIG_3 = new PresetConfig(
            "config_3",
            "Configuración 3 (Capas Cilíndricas - Eje Y)",
            "2*x", "x^2",
            0, 2, Metodo.CAPAS, Eje.Y,
            8.3776, // 2*pi * int_0^2 x*(2x - x^2) dx = 2*pi*(4/3) = 8*pi/3 = 8.37758...
            "f(x) = 2x, g(x) = x^2 en [0, 2] rotado en el Eje Y."
    );

    public static final List<PresetConfig> ALL = List.of(MAIN_PROJECT, CONFIG_1, CONFIG_2, CONFIG_3);
}
```

engine/FunctionEvaluator.java

```
package com.umg.calculo.engine;


import net.objecthunter.exp4j.Expression;
import net.objecthunter.exp4j.ExpressionBuilder;


import java.util.function.DoubleUnaryOperator;


/**
 * Compiles a user-typed expression like "x^2 + 1" or "sqrt(x)" into something
 * that can be evaluated at any x. Evaluation failures (domain errors, e.g.
 * sqrt of a negative number) return NaN instead of throwing, so the caller's
 * validation logic can detect and report them uniformly.
 */
public final class FunctionEvaluator implements DoubleUnaryOperator {


    private final Expression expression;
    private final String rawExpression;


    private FunctionEvaluator(Expression expression, String rawExpression) {
        this.expression = expression;
        this.rawExpression = rawExpression;
    }


    /** @throws IllegalArgumentException if the expression has invalid syntax */
    public static FunctionEvaluator compile(String expr) {
        try {
            Expression e = new ExpressionBuilder(expr).variable("x").build();
            return new FunctionEvaluator(e, expr);
        } catch (Exception ex) {
```

```
            throw new IllegalArgumentException("Expresión inválida: " + expr, ex);
        }
    }

    @Override
    public double applyAsDouble(double x) {
        try {
            expression.setVariable("x", x);
            double result = expression.evaluate();
            return Double.isFinite(result) ? result : Double.NaN;
        } catch (Exception ex) {
            return Double.NaN;
        }
    }

    public String rawExpression() {
        return rawExpression;
    }

    /**
     * Evaluates a constant typed by the user (limits a, b or the manual
     * volume). Besides plain numbers it accepts a decimal comma ("73,5133"),
     * pi ("117*pi/5", "23.4π") and functions ("sqrt(2)"), so a value copied
     * straight from the hand-written solution can be pasted as-is.
     * Returns NaN if the text is empty, invalid or depends on x.
     */
    public static double evaluateConstant(String text) {
        if (text == null || text.isBlank()) return Double.NaN;
        String s = text.trim().replace(',', '.').replace("π", "pi");
        try {
            double v = new ExpressionBuilder(s).build().evaluate();
            return Double.isFinite(v) ? v : Double.NaN;
        } catch (Exception ex) {
            return Double.NaN;
        }
    }
}
```

engine/Validator.java

```
package com.umg.calculo.engine;

import com.umg.calculo.model.Eje;
import com.umg.calculo.model.Metodo;

import java.util.ArrayList;
import java.util.List;
import java.util.function.DoubleUnaryOperator;

/** Data-entry validation rules from the assignment (Section 6). */
public final class Validator {

    private Validator() {}

    public record Result(List<String> errores) {
        public boolean esValido() {
            return errores.isEmpty();
        }
    }

    public static Result validar(double a, double b, Metodo metodo, Eje eje,
                                 DoubleUnaryOperator f, DoubleUnaryOperator g) {
        List<String> errores = new ArrayList<>();

        if (Double.isNaN(a) || Double.isNaN(b)) {
            errores.add("Los límites de integración a y b deben ser números válidos.");
        } else if (a >= b) {
            errores.add("Debe cumplirse que a < b.");
```

```
        }

        if ((metodo == Metodo.DISCOS || metodo == Metodo.ARANDELAS) && eje != Eje.X) {
            errores.add("Los métodos de discos y arandelas deben ser alrededor del eje X.");
        }
        if (metodo == Metodo.CAPAS && eje != Eje.Y) {
            errores.add("El método de capas cilíndricas debe ser alrededor del eje Y.");
        }
        if (metodo == Metodo.CAPAS && a < 0) {
            errores.add("Para capas cilíndricas alrededor del eje Y debe cumplirse 0 <= a < b.");
        }

        if (!Double.isNaN(a) && !Double.isNaN(b) && a < b) {
            int steps = 200;
            double stepSize = (b - a) / steps;
            boolean invalidFunc = false;
            boolean lowerViolated = false;
            boolean negViolated = false;

            for (int i = 0; i <= steps; i++) {
                double x = a + i * stepSize;
                double fx = f.applyAsDouble(x);
                double gx = g.applyAsDouble(x);

                if (Double.isNaN(fx) || Double.isNaN(gx)) {
                    invalidFunc = true;
                    break;
                }
                if (fx < gx - 1e-9) {
                    lowerViolated = true;
                }
                if ((metodo == Metodo.DISCOS || metodo == Metodo.ARANDELAS) && gx < -1e-9) {
                    negViolated = true;
                }
            }

            if (invalidFunc) {
                errores.add("Las funciones no se pudieron evaluar en todo el intervalo [a, b].");
            }
            if (lowerViolated) {
                errores.add("Debe cumplirse f(x) >= g(x) en todo el intervalo [a, b].");
            }
            if (negViolated) {
                errores.add("Para discos y arandelas, debe cumplirse f(x) >= g(x) >= 0 en [a, b].");
            }
        }

        return new Result(errores);
    }
}
```

engine/VolumeCalculator.java

```
package com.umg.calculo.engine;

import com.umg.calculo.model.Metodo;

import java.util.function.DoubleUnaryOperator;

/**
 * Numerical core: Simpson's 1/3 rule integration, and the volume integrand
 * for each of the three methods studied in the course. This is the piece the
 * assignment (Section 7) requires to match the manual derivation exactly.
 */
public final class VolumeCalculator {

    private VolumeCalculator() {}
```

```
    /** Composite Simpson's rule on [a, b] with n subintervals (bumped up to even if odd). */
    public static double simpsonIntegrate(DoubleUnaryOperator fn, double a, double b, int n) {
        if (a >= b) return 0.0;
        if (n % 2 != 0) n++;
        double h = (b - a) / n;
        double sum = fn.applyAsDouble(a) + fn.applyAsDouble(b);

        for (int i = 1; i < n; i++) {
            double x = a + i * h;
            double coef = (i % 2 == 0) ? 2.0 : 4.0;
            sum += coef * fn.applyAsDouble(x);
        }
        return (h / 3.0) * sum;
    }

    public static double simpsonIntegrate(DoubleUnaryOperator fn, double a, double b) {
        return simpsonIntegrate(fn, a, b, 800);
    }

    /**
     * Builds V(x)'s integrand for the given method:
     *  - Discos:    V = pi * int [f(x)]^2 dx
     *  - Arandelas: V = pi * int ([f(x)]^2 - [g(x)]^2) dx
     *  - Capas:     V = 2*pi * int x * (f(x) - g(x)) dx
     */
    public static DoubleUnaryOperator buildIntegrand(DoubleUnaryOperator f, DoubleUnaryOperator g, Metodo
metodo) {
        return switch (metodo) {
            case DISCOS -> x -> Math.PI * Math.pow(f.applyAsDouble(x), 2);
            case ARANDELAS -> x -> Math.PI * (Math.pow(f.applyAsDouble(x), 2) - Math.pow(g.applyAsDouble(x),
2));
            case CAPAS -> x -> 2 * Math.PI * x * (f.applyAsDouble(x) - g.applyAsDouble(x));
        };
    }

    /** V_acum(x) = volumen generado desde a hasta x (Section 11). */
    public static double accumulatedVolume(DoubleUnaryOperator integrand, double a, double x) {
        return simpsonIntegrate(integrand, a, x, 200);
    }
}
```

engine/IntersectionFinder.java

```
package com.umg.calculo.engine;

import java.util.ArrayList;
import java.util.List;
import java.util.function.DoubleUnaryOperator;

/**
 * Finds the intersection points of f(x) and g(x), i.e. the roots of
 * h(x) = f(x) - g(x) (part "a" of the main exercise). Scans [lo, hi] on a fine
 * grid looking for sign changes, then refines each one by bisection. Points
 * where f or g are undefined (NaN, e.g. sqrt of a negative) are skipped.
 */
public final class IntersectionFinder {

    public static final double DEFAULT_LO = -10;
    public static final double DEFAULT_HI = 10;
    private static final int SAMPLES = 4000;
    private static final double ZERO_TOL = 1e-12;

    private IntersectionFinder() {}

    /** Region between two consecutive intersections where f(x) >= g(x). */
    public record Interval(double a, double b) {}

    public static List<Double> findIntersections(DoubleUnaryOperator f, DoubleUnaryOperator g) {
```

```
        return findRoots(x -> f.applyAsDouble(x) - g.applyAsDouble(x), DEFAULT_LO, DEFAULT_HI, SAMPLES);
    }

    public static List<Double> findRoots(DoubleUnaryOperator h, double lo, double hi, int samples) {
        double step = (hi - lo) / samples;
        List<Double> roots = new ArrayList<>();
        double prevX = Double.NaN, prevH = Double.NaN;

        for (int i = 0; i <= samples; i++) {
            // lo + (hi - lo) * i / samples (not lo + i*step) so grid points like
            // x = 0 land exactly, which catches roots at a NaN boundary (sqrt(x) at 0).
            double x = lo + (hi - lo) * i / samples;
            double hx = h.applyAsDouble(x);
            if (Double.isNaN(hx)) { prevH = Double.NaN; continue; }

            if (Math.abs(hx) < ZERO_TOL) {
                addRoot(roots, x, step);
            } else if (!Double.isNaN(prevH) && Math.abs(prevH) >= ZERO_TOL && Math.signum(prevH) !=
Math.signum(hx)) {
                addRoot(roots, bisect(h, prevX, x), step);
            }
            prevX = x;
            prevH = hx;
        }
        return roots;
    }

    /**
     * First pair of consecutive intersections with f above g between them,
     * or null if there is none (fewer than two intersections, or g above f).
     */
    public static Interval regionBetween(List<Double> roots, DoubleUnaryOperator f, DoubleUnaryOperator g) {
        for (int i = 0; i + 1 < roots.size(); i++) {
            double r1 = roots.get(i), r2 = roots.get(i + 1);
            double mid = (r1 + r2) / 2;
            if (f.applyAsDouble(mid) > g.applyAsDouble(mid)) return new Interval(r1, r2);
        }
        return null;
    }

    private static double bisect(DoubleUnaryOperator h, double lo, double hi) {
        double hLo = h.applyAsDouble(lo);
        for (int i = 0; i < 100; i++) {
            double mid = (lo + hi) / 2;
            double hMid = h.applyAsDouble(mid);
            if (hMid == 0) return mid;
            if (Math.signum(hMid) == Math.signum(hLo)) { lo = mid; hLo = hMid; } else { hi = mid; }
        }
        return (lo + hi) / 2;
    }

    /** Skips near-duplicates (a flat h can be ~0 on several adjacent grid points). */
    private static void addRoot(List<Double> roots, double x, double step) {
        if (!roots.isEmpty() && x - roots.get(roots.size() - 1) < 1.5 * step) return;
        roots.add(x);
    }
}
```

engine/RevolutionMeshBuilder.java

```
package com.umg.calculo.engine;

import com.umg.calculo.model.Eje;
import com.umg.calculo.model.Metodo;
import javafx.scene.shape.TriangleMesh;
import javafx.scene.shape.VertexFormat;

import java.util.ArrayList;
import java.util.List;
```

```
import java.util.function.DoubleUnaryOperator;

/**
 * Builds the solid-of-revolution triangle mesh from a single CLOSED 2D
 * profile (radius, height) that is swept around the axis. Because the
 * profile itself traces outer curve -> end cap -> inner curve -> end cap
 * back to the start (a closed loop), sweeping it a full turn produces one
 * watertight solid — no separate cap/wall pieces needed.
 *
 * Convention (matches the coordinate mapping used by the calculus methods):
 *  - X-axis (discos/arandelas): radius = curve value, height = x.
 *  - Y-axis (capas): radius = x, height = curve value.
 */
public final class RevolutionMeshBuilder {

    private static final int PROFILE_SAMPLES = 60;

    private RevolutionMeshBuilder() {}

    /** One (radius, height) point of the profile, tagged with which curve (f or g) it belongs to. */
    public record ProfilePoint(double r, double h, boolean isCurveF) {}

    public static List<ProfilePoint> buildProfile(DoubleUnaryOperator f, DoubleUnaryOperator g,
                                                  double a, double b, Metodo metodo, Eje eje) {
        int n = PROFILE_SAMPLES;
        List<ProfilePoint> pts = new ArrayList<>(2 * n + 2);

        if (eje == Eje.X) {
            DoubleUnaryOperator rOuter = x -> Math.max(0, f.applyAsDouble(x));
            DoubleUnaryOperator rInner = x -> metodo == Metodo.ARANDELAS ? Math.max(0, g.applyAsDouble(x)) :
0.0;

            pts.add(new ProfilePoint(rInner.applyAsDouble(a), a, false));
            pts.add(new ProfilePoint(rOuter.applyAsDouble(a), a, true));
            for (int i = 1; i <= n; i++) {
                double x = a + (i / (double) n) * (b - a);
                pts.add(new ProfilePoint(rOuter.applyAsDouble(x), x, true));
            }
            pts.add(new ProfilePoint(rInner.applyAsDouble(b), b, false));
            for (int i = n - 1; i >= 1; i--) {
                double x = a + (i / (double) n) * (b - a);
                pts.add(new ProfilePoint(rInner.applyAsDouble(x), x, false));
            }
        } else {
            for (int i = 0; i <= n; i++) {
                double x = a + (i / (double) n) * (b - a);
                pts.add(new ProfilePoint(x, g.applyAsDouble(x), false));
            }
            pts.add(new ProfilePoint(b, f.applyAsDouble(b), true));
            for (int i = n - 1; i >= 1; i--) {
                double x = a + (i / (double) n) * (b - a);
                pts.add(new ProfilePoint(x, f.applyAsDouble(x), true));
            }
            pts.add(new ProfilePoint(a, f.applyAsDouble(a), true));
        }
        return pts;
    }

    /**
     * @param sweepFraction (0, 1]: 1 = full solid; less = partial reveal, used
     *                      by the sweep-generation animation.
     * @param radialSegments mesh resolution around the axis of revolution.
     */
    public static TriangleMesh buildMesh(List<ProfilePoint> profile, double sweepFraction, int radialSegments)
{
        int nPts = profile.size();
        double phiLength = 2 * Math.PI * Math.max(0.001, Math.min(1, sweepFraction));

        float[] points = new float[nPts * (radialSegments + 1) * 3];
```

```
        float[] texCoords = new float[nPts * (radialSegments + 1) * 2];

        int pIdx = 0, tIdx = 0;
        for (int j = 0; j <= radialSegments; j++) {
            double theta = (j / (double) radialSegments) * phiLength;
            float cosT = (float) Math.cos(theta);
            float sinT = (float) Math.sin(theta);
            for (ProfilePoint pt : profile) {
                points[pIdx++] = (float) (pt.r() * cosT);
                points[pIdx++] = (float) pt.h();
                points[pIdx++] = (float) (pt.r() * sinT);

                // Samples a 2-color strip texture: near 0.25 = f(x) color,
                // near 0.75 = g(x) color (see Solid3DView's material texture).
                texCoords[tIdx++] = pt.isCurveF() ? 0.25f : 0.75f;
                texCoords[tIdx++] = 0.5f;
            }
        }

        int quadsPerRing = nPts - 1;
        int[] faces = new int[radialSegments * quadsPerRing * 12]; // 2 triangles * 3 verts * 2
(point+texcoord) idx
        int fIdx = 0;
        for (int j = 0; j < radialSegments; j++) {
            for (int i = 0; i < quadsPerRing; i++) {
                int a0 = j * nPts + i;
                int a1 = j * nPts + i + 1;
                int b0 = (j + 1) * nPts + i;
                int b1 = (j + 1) * nPts + i + 1;

                faces[fIdx++] = a0; faces[fIdx++] = a0;
                faces[fIdx++] = b0; faces[fIdx++] = b0;
                faces[fIdx++] = a1; faces[fIdx++] = a1;

                faces[fIdx++] = a1; faces[fIdx++] = a1;
                faces[fIdx++] = b0; faces[fIdx++] = b0;
                faces[fIdx++] = b1; faces[fIdx++] = b1;
            }
        }

        TriangleMesh mesh = new TriangleMesh(VertexFormat.POINT_TEXCOORD);
        mesh.getPoints().setAll(points);
        mesh.getTexCoords().setAll(texCoords);
        mesh.getFaces().setAll(faces);
        return mesh;
    }

    /** Bounding info used to center the mesh and frame the camera. */
    public record Bounds(double centerH, double maxExtent) {}

    public static Bounds computeBounds(List<ProfilePoint> profile) {
        double minH = Double.POSITIVE_INFINITY, maxH = Double.NEGATIVE_INFINITY, maxR = 0;
        for (ProfilePoint p : profile) {
            minH = Math.min(minH, p.h());
            maxH = Math.max(maxH, p.h());
            maxR = Math.max(maxR, Math.abs(p.r()));
        }
        double centerH = (minH + maxH) / 2.0;
        double halfHeight = (maxH - minH) / 2.0;
        double maxExtent = Math.max(maxR, halfHeight);
        return new Bounds(centerH, Math.max(maxExtent, 0.5));
    }
}
```

export/DataExporter.java

```
package com.umg.calculo.export;

import java.io.IOException;
```

```
import java.io.Writer;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.Locale;

/** Exports the x, f(x), g(x), V_acum(x) sample data required by Section 12. */
public final class DataExporter {

    private DataExporter() {}

    public record Row(double x, double fx, double gx, double vAcum) {}

    public static void exportCsv(Path file, List<Row> rows) throws IOException {
        try (Writer w = Files.newBufferedWriter(file, StandardCharsets.UTF_8)) {
            w.write("x,f_x,g_x,V_acum\n");
            for (Row r : rows) {
                w.write(fmt(r.x()) + "," + fmt(r.fx()) + "," + fmt(r.gx()) + "," + fmt(r.vAcum()) + "\n");
            }
        }
    }

    public static void exportTxt(Path file, List<Row> rows) throws IOException {
        try (Writer w = Files.newBufferedWriter(file, StandardCharsets.UTF_8)) {
            w.write("x\tf(x)\tg(x)\tV_acum\n");
            for (Row r : rows) {
                w.write(fmt(r.x()) + "\t" + fmt(r.fx()) + "\t" + fmt(r.gx()) + "\t" + fmt(r.vAcum()) + "\n");
            }
        }
    }

    private static String fmt(double v) {
        return String.format(Locale.US, "%.4f", v);
    }
}
```

export/ImageExporter.java

```
package com.umg.calculo.export;

import javafx.scene.image.Image;
import javafx.scene.image.PixelReader;
import javafx.scene.image.WritablePixelFormat;

import javax.imageio.ImageIO;
import java.awt.image.BufferedImage;
import java.io.IOException;
import java.nio.file.Path;

/**
 * Saves a JavaFX image as PNG (Section 17: graphs delivered as PNG). Copies the
 * pixels into an AWT BufferedImage so it doesn't need the extra javafx-swing module.
 */
public final class ImageExporter {

    private ImageExporter() {}

    public static void savePng(Image image, Path file) throws IOException {
        int w = (int) image.getWidth(), h = (int) image.getHeight();
        int[] argb = new int[w * h];
        PixelReader reader = image.getPixelReader();
        reader.getPixels(0, 0, w, h, WritablePixelFormat.getIntArgbInstance(), argb, 0, w);

        BufferedImage out = new BufferedImage(w, h, BufferedImage.TYPE_INT_ARGB);
        out.setRGB(0, 0, w, h, argb, 0, w);
        if (!ImageIO.write(out, "png", file.toFile())) {
```

```
            throw new IOException("No hay un escritor PNG disponible.");
        }
    }
}
```

Anexo B. Capturas de pantalla del simulador

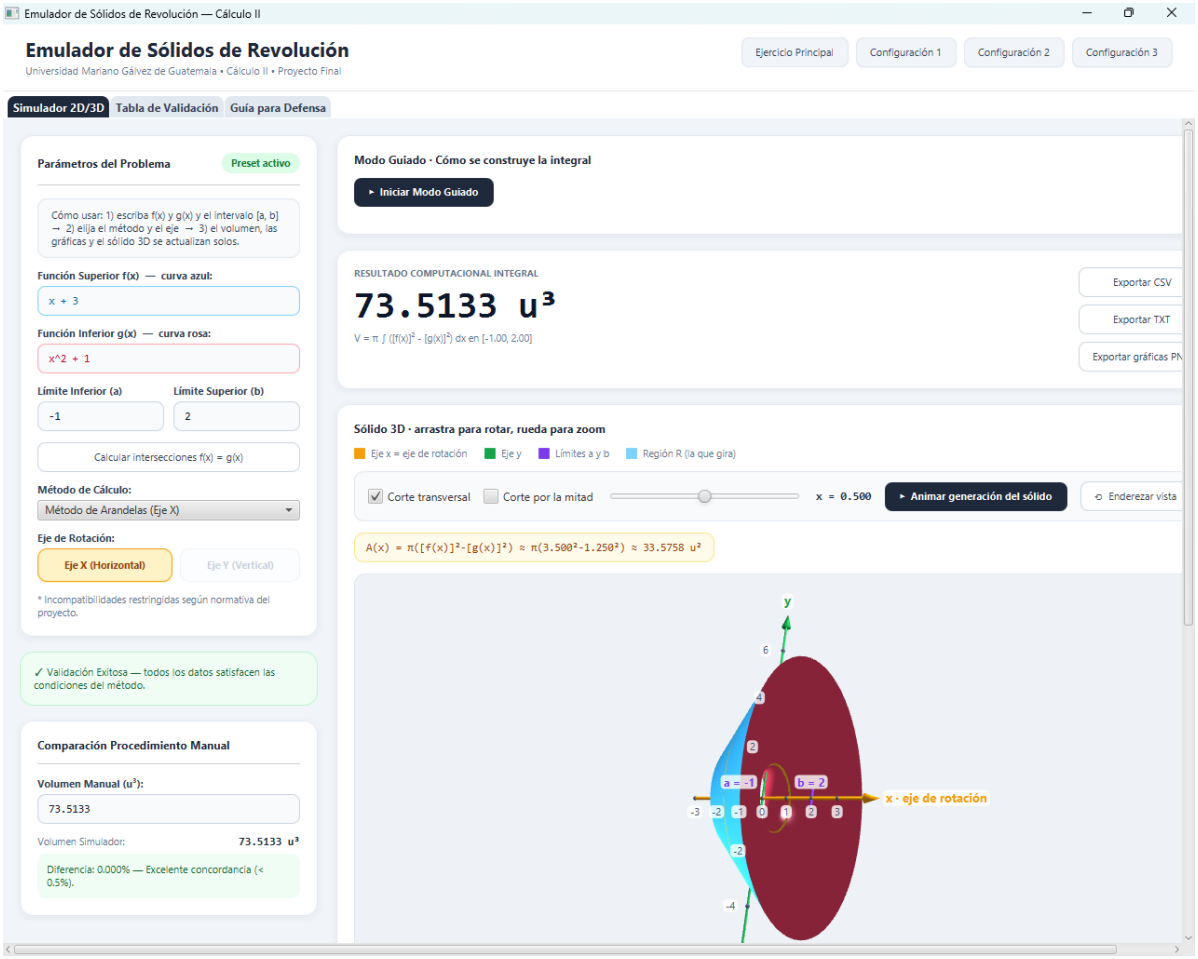


Figura 13. *Ejercicio principal: parámetros, validación exitosa, volumen 73.5133 u³, comparación manual y sólido 3D con corte transversal.*

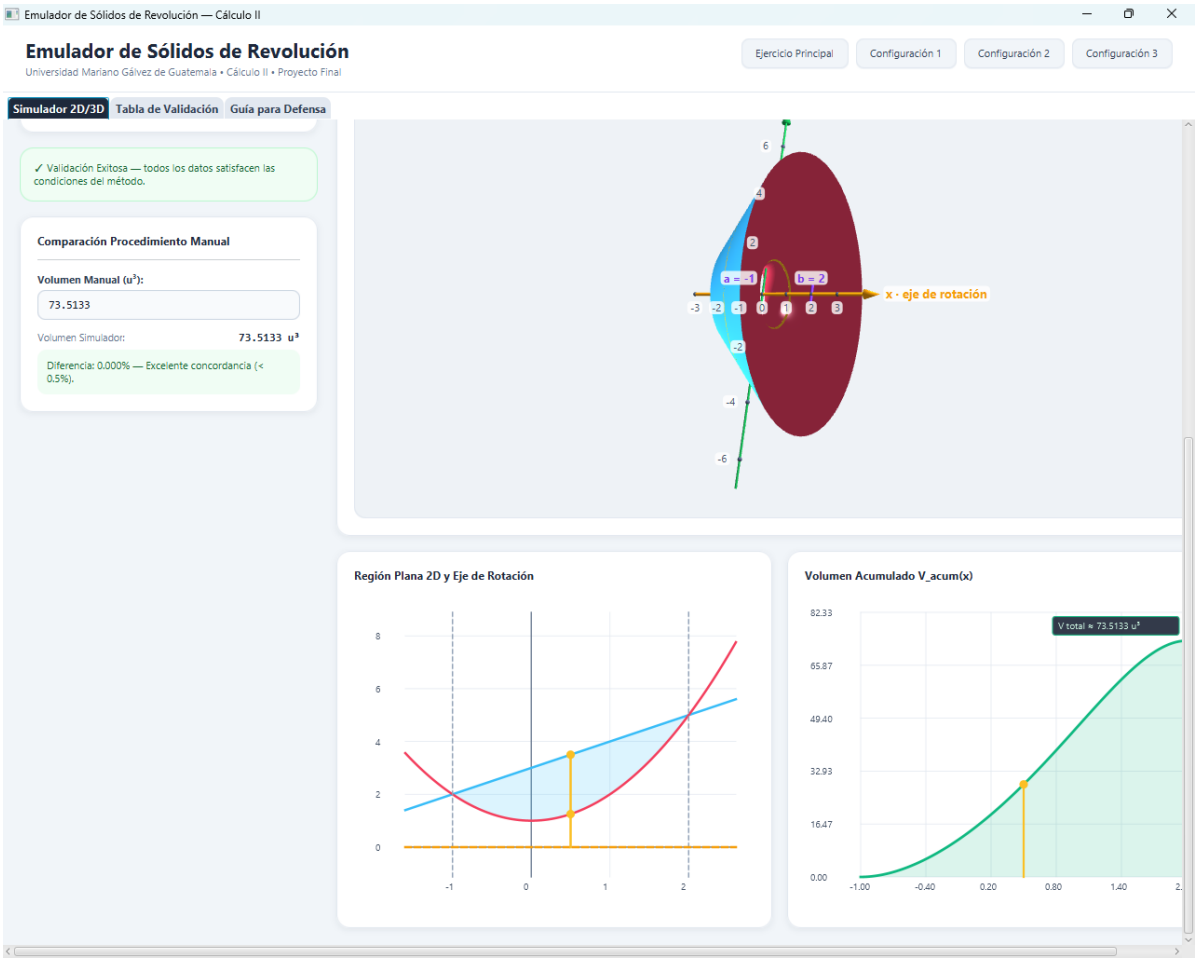
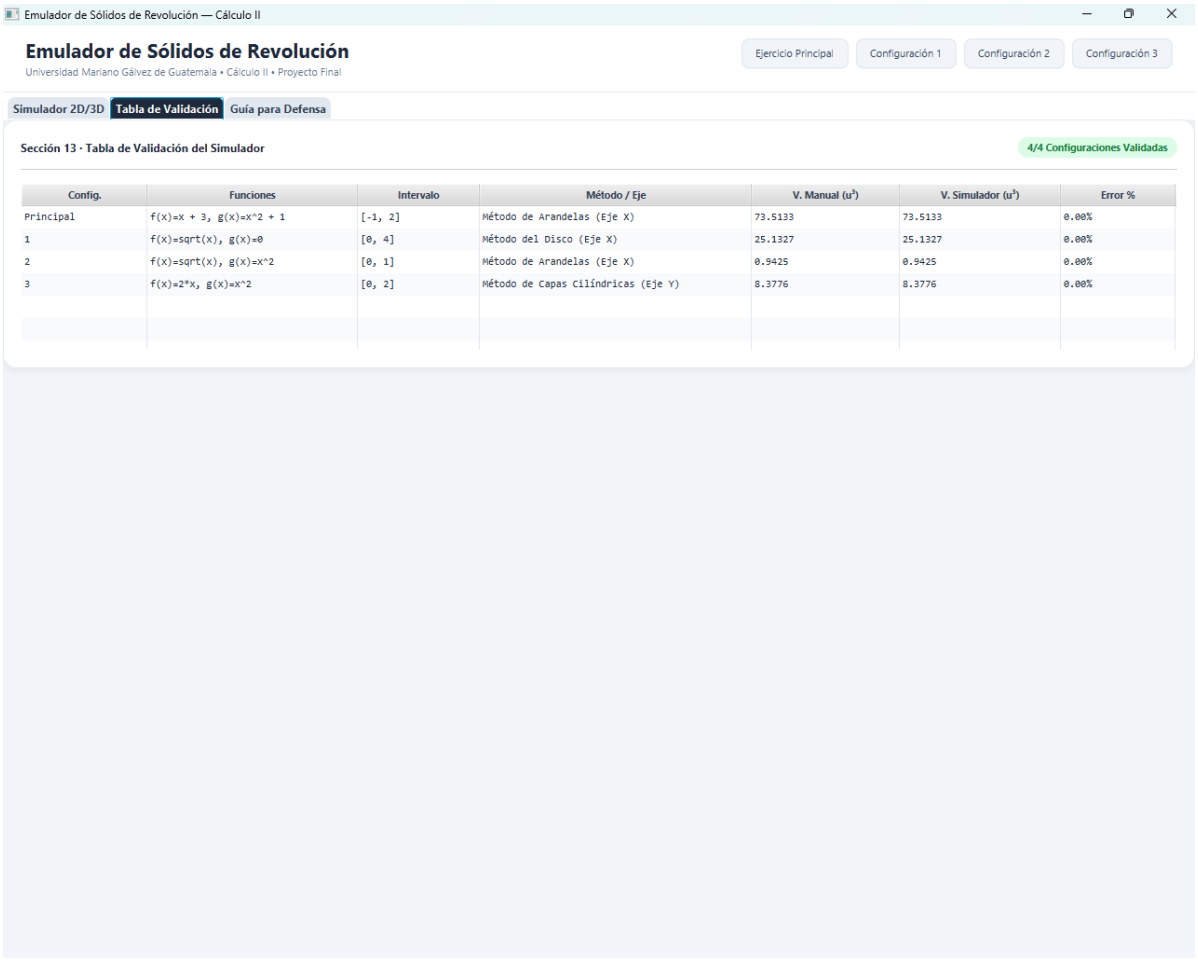


Figura 14. *Gráfica 2D de la región con el eje de rotación y gráfica del volumen acumulado V_acum(x).*



Config.	Funciones	Intervalo	Método / Eje	V. Manual (u^3)	V. Simulador (u^3)	Error %
Principal	f(x)=x + 3, g(x)=x^2 + 1	[-1, 2]	Método de Arandelas (Eje X)	73.5133	73.5133	0.00%
1	f(x)=sqrt(x), g(x)=0	[0, 4]	Método del Disco (Eje X)	25.1327	25.1327	0.00%
2	f(x)=sqrt(x), g(x)=x^2	[0, 1]	Método de Arandelas (Eje X)	0.9425	0.9425	0.00%
3	f(x)=2*x, g(x)=x^2	[0, 2]	Método de Capas Cilíndricas (Eje Y)	8.3776	8.3776	0.00%

Figura 15. *Pestaña «Tabla de Validación»: 4/4 configuraciones validadas.*

Anexo C. Archivo de datos exportado

Archivo datos_solido.csv generado para el ejercicio principal (151 filas). Se muestran las primeras y las últimas filas; el archivo completo se entrega junto con el informe.

x	f_x	g_x	V_acum
-1.0000	2.0000	2.0000	0.0000
-0.9800	2.0200	1.9604	0.0075
-0.9600	2.0400	1.9216	0.0297
-0.9400	2.0600	1.8836	0.0663
-0.9200	2.0800	1.8464	0.1170
-0.9000	2.1000	1.8100	0.1815
-0.8800	2.1200	1.7744	0.2594
-0.8600	2.1400	1.7396	0.3505
-0.8400	2.1600	1.7056	0.4545
-0.8200	2.1800	1.6724	0.5712
-0.8000	2.2000	1.6400	0.7002
…	…	…	…
1.9000	4.9000	4.6100	73.0676
1.9200	4.9200	4.6864	73.2248
1.9400	4.9400	4.7636	73.3492
1.9600	4.9600	4.8416	73.4395
1.9800	4.9800	4.9204	73.4946
2.0000	5.0000	5.0000	73.5133

Anexo D. Gráficas

Las gráficas de la región (Figura 2), del sólido 3D (Figuras 3 y 4), de las configuraciones de validación (Figuras 5 a 11) y del volumen acumulado (Figura 12) se incluyen en el cuerpo del informe y se entregan también como archivos PNG en la carpeta graficas.